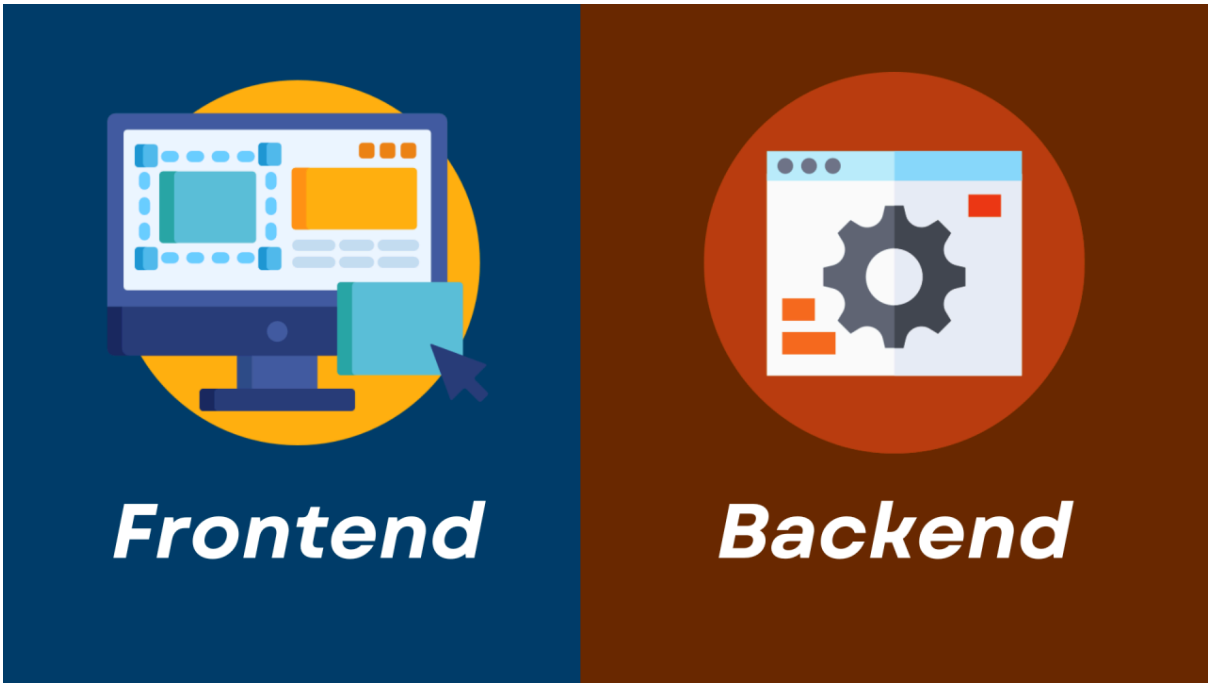
OP1-B1-01-API HTTP



Esteve Amargant Blanco
Maria Pilar Moreno
Frameworks de forntend i backends

BLOC A. CONCEPTES BÀSICS (Escalfament)

Exercici A1
Escriu amb MÀXIM 1 línia cadascun:

a) API: Una API et descriu que pots fer i com fer-ho

b) HTTP: El que utilitza el motor de cerca per comunicar-se amb els servidors

c) REST: Directrius per construir API's

d) Recurs: Un element que es pot accedir o utilitzar.

e) Endpoint: Ruta en una API que fa X coses, depenen del verb del endpoint.

Exercici A2

Uneix cada verb HTTP amb la descripció (posa la lletra):

1. GET 2. POST 3. PUT 4. PATCH 5. DELETE

a) Eliminar un recurs → DELETE
b) Crear un recurs nou → POST
c) Llegir dades → GET
d) Actualitzar parcialment → PATCH
e) Substituir totalment → PUT

Exercici A3

Quina opció és més "REST" per crear un producte?

a) **GET /crearProducte?nom=A** → AQUESTA
b) POST /productes
c) POST /crearProducte
d) PUT /productes/crear

Exercici A4 (Veritat / Fals – 4')

- Cada petició HTTP inclou sempre un body. → Fals
- Un 404 vol dir que el servidor està caigut. → Fals
- REST obliga a fer servir JSON. → Fals
- Un 201 indica que s'ha creat un recurs nou. → Cert, indica que ha s'ha completat la peticio correctament.
- PATCH i PUT són exactament iguals. → Fals

BLOC B. DISSENY D'ENDPOINTS

Context: Sistema Biblioteca amb recursos: llibres, usuaris, préstecs.

Exercici B1

Defineix MÈTODE + PATH per:

- Llistar tots els llibres
    - GET /llibres
- Afegir un llibre nou
    - POST /llibres
- Consultar dades del llibre amb id 90
    - GET /llibres/90
- Eliminar el llibre 77
    - DELETE /llibres/77
- Actualitzar només el títol del llibre 12
    - PATCH /llibres/12/{titol:noutitol}
- Substituir completament dades del llibre 12
    - PUT /llibres/12

BLOC C. ANALITZAR I CORREGIR
Exercici C1 (Detecta 5 errors)

Petició proposada:

```
POST /crearLlibre/ HTTP/1.1
Host: api.biblio.local
ContentType: text/plain
Body:
{ titol: "DDD", "autor": "Evans" }
```

Quins errors hi veus? Escriu-los i proposa correcció.

```
POST /crearLlibre/ HTTP/1.1
```

Sobra el crear del crearLlibre, aixo ja tho "diu" el POST i la barra / del final tambe ContentType ha de ser Content-Type i diu que es text pla pero surt en format format JSON i la etiqueta Body no hi hauria de anar tampoc. A part en el JSON, titol hauria de ser "titol"

Exercici C2 (Resposta qüestionable)

Resposta rebuda:

```
HTTP/1.1 200 OK
Content-Type: application/json
{ "error": "Llibre no trobat" }
```

Quin codi d'estat hauria de ser? Per què?
Si es un error ha de ser un codi 400 minim, i si es per que el llibre no ha sigut trobat, un 404 Not Found

Exercici C3 (Req vs Resp – 5')

Marca quin camp pertany a Request (R), quin a Response (S):

`Status Code` Response (S)
`Body amb JSON enviat pel client` Request (R)
`Authorization Header` Request (R)
`Content-Type: application/json (del que torna el servidor)` Response (S)
`Accept: application/json` Request ®

BLOC D. SEQÜÈNCIES DE PETICIONS
Exercici D1 (Orde lògic – 6')

Ordena (1→5) les accions per signar-se i veure perfil:

_5_ GET /usuaris/jo (requereix estar autenticat)

_1_ POST /auth/login (rep token)

_2_ Guardar el token localment

_3_ GET /auth/status (opcional, comprova token)

_4_ Fer fetch amb Header Authorization: Bearer <token>

Exercici D2 (Pensar l'estat – 5')

Per què el servidor "no recorda" que ja hem fet login si no enviem el token? Explica-ho en 2 frases (paraula clau: stateless).

Per que cada petició es processa de forma independent i el servidor no guarda cap memòria de peticions anteriors del mateix client. **HHTP → Stateless**

BLOC E. CURL / FETCH
Exercici E1 (curl)

Escriu un curl per: GET https://api.exemple.com/usuaris acceptant JSON.

```
curl -X GET https://api.exemple.com/usuaris \
  -H "Accept: application/json"
```

Exercici E2 (curl amb body)

Crear usuari amb email "ana@test.com" i nom "Ana": endpoint POST /usuaris a https://api.exemple.com. Afegeix header correcte.

```
curl -X POST https://api.exemple.com/usuaris \
  -H "Content-Type: application/json" \
  -d '{"email": "ana@test.com", "nom": "Ana"}'
```

Exercici E3 (fetch)

Escriu codi JavaScript (fetch) per fer GET /llibres i mostrar per consola el nombre total (suposa que la resposta és un array JSON).

```
fetch('/llibres')
  .then(response => response.json())
  .then(llibres => console.log(llibres.length));
```

BLOC H. REPÀS DE CODIS D'ESTAT
Exercici H1 (Classifica)

Assigna descripció a codi: 200, 201, 400, 401, 403, 404, 500

a) No trobat → 404

b) Creat → 201

c) Correcte → 200

d) Error servidor → 500

e) Falta autenticació → 401

f) Prohibit (sense permís) → 403

g) Petició mal construïda → 400

Exercici H2 (Justifica)

Per què no és bona idea retornar sempre 200 i posar l'error dins del JSON? (2 raons)

Per que s'han de seguir unes pautes i per algo estan els codis de errors, i per que els propis nvagadors llegeixen els mateixos codis de estat per saber com ha anat la petició

BLOC G. MINI CAS D'ÚS (PROJECTE EXEMPLE)
Context: API de reserves de sales. Recursos: usuaris, sales, reserves.

Acció: Crear una reserva autenticada.

Exercici G1 (Seqüència)

Descriu (llista numerada) les peticions necessàries des de "usuari nou" fins "reserva creada". Inclou verbs i paths.

1. POST /auth/register
2. POST /auth/login
3. GET /sales
4. GET /sales/{id}
5. POST /reserves

Exercici G2 (Error esperat)

Si intento reservar una sala inexistent (id 9999), quin codi d'estat i missatge curt retornaries?

404 Not Found
{ "error": "Sala 9999 no trobada" }